

Redmine MCP 및 Codex·Claude Code 로컬 플러그인 요구사항

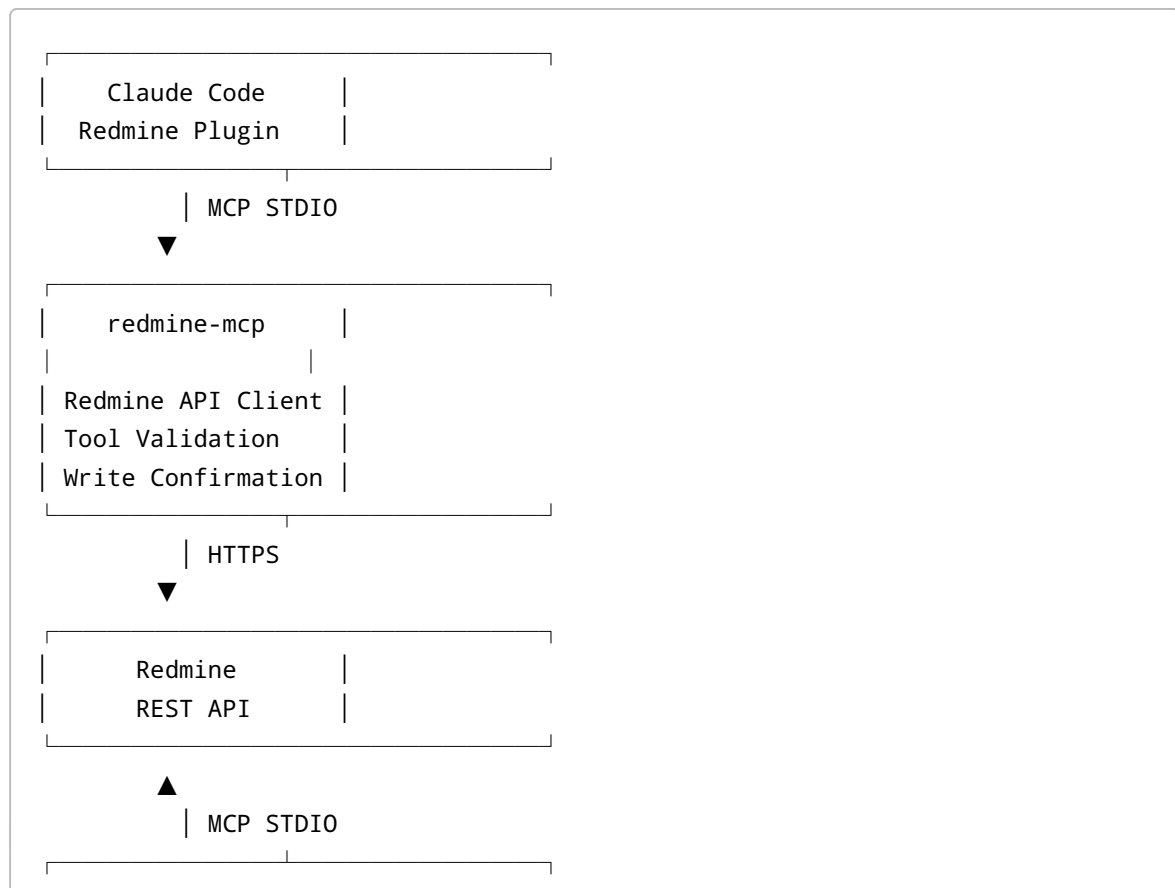
1. 문서 개요

1.1 목적

본 문서는 다음 세 가지 구성요소의 개발 요구사항을 정의한다.

1. `redmine-mcp`
2. Redmine REST API를 MCP 도구로 제공하는 공통 서버
3. Claude Code 로컬 플러그인
4. Claude Code에서 `redmine-mcp` 를 쉽게 설치하고 사용하기 위한 플러그인
5. Codex 로컬 플러그인
6. Codex에서 `redmine-mcp` 를 쉽게 설치하고 사용하기 위한 플러그인

1.2 기본 구조



Codex	
Redmine Plugin	

Codex와 Claude Code 모두 로컬 프로세스로 실행되는 STUDIO 방식의 MCP 서버를 사용할 수 있다. Codex 플러그인은 `.codex-plugin/plugin.json`, `.mcp.json`, `skills/` 등을 포함할 수 있으며, Claude Code 플러그인은 `.claude-plugin/plugin.json`, `.mcp.json`, `skills/` 등을 포함할 수 있다.

2. 개발 목표

2.1 핵심 목표

사용자는 Codex 또는 Claude Code에서 자연어로 다음 작업을 수행할 수 있어야 한다.

Cloud HMI 프로젝트에서 내게 할당된 열린 이슈를 보여줘.

#1523 이슈의 상세 내용과 최근 댓글을 보여줘.

현재 Git 변경사항을 기반으로 Redmine 이슈 초안을 작성해줘.

#1523 이슈에 구현 내용을 댓글로 추가해줘.

#1523 이슈를 완료 상태로 변경해줘.

2.2 설계 원칙

- Redmine 관련 비즈니스 로직은 `redmine-mcp`에 집중한다.
- Codex와 Claude Code 플러그인은 최대한 얇게 구성한다.
- 조회 작업과 쓰기 작업을 명확히 구분한다.
- 쓰기 작업은 사용자의 명시적인 승인 없이 실행하지 않는다.
- Redmine API Key는 코드, 저장소, 로그에 저장하지 않는다.
- Redmine에 설정된 기존 사용자 권한을 그대로 따른다.
- Redmine 서버에 별도의 Ruby 플러그인을 설치하지 않아도 동작해야 한다.

3. 대상 사용자

3.1 일반 개발자

다음 작업을 수행한다.

- 담당 이슈 조회
- 이슈 상세 조회
- 작업할 이슈 확인
- 댓글 추가

- 상태 변경
- 작업 내용을 바탕으로 신규 이슈 등록

3.2 프로젝트 관리자

다음 작업을 수행할 수 있다.

- 프로젝트 이슈 검색
- 담당자 지정
- 우선순위 변경
- 목표 버전 지정
- 이슈 상태 변경

단, 실제 가능한 작업 범위는 사용자의 Redmine 권한에 따라 제한되어야 한다.

3.3 시스템 관리자

다음 항목을 관리한다.

- 사내 npm 레지스트리 배포
- 플러그인 배포 버전
- 허용된 Redmine 서버 주소
- 최소 지원 버전
- 보안 정책
- MCP 서버 허용 정책

4. 사전 요구사항

4.1 Redmine 요구사항

- Redmine REST API가 활성화되어 있어야 한다.
- 사용자는 Redmine API Key를 발급받을 수 있어야 한다.
- Redmine URL은 사용자 PC에서 접근 가능해야 한다.
- 사내망 Redmine인 경우 사용자는 VPN 또는 사내 네트워크에 연결되어 있어야 한다.
- HTTPS 사용을 원칙으로 한다.
- 사설 인증서를 사용할 경우 CA 인증서 파일을 배포해야 한다.

Redmine REST API는 관리 설정에서 활성화해야 하며 API Key를 `X-Redmine-API-Key` 헤더로 전달할 수 있다. 목록 API는 기본 25개, 최대 100개 단위의 페이지네이션을 사용한다.

4.2 로컬 실행 환경

필수

- Windows 10/11, macOS 또는 Linux
- Node.js 20 이상
- npm, pnpm 또는 사내 npm 레지스트리 접근
- Claude Code 최신 안정 버전

- Codex 최신 안정 버전

권장

- Node.js 22 LTS
- Git
- PowerShell 7 또는 Bash
- 회사 VPN
- 사내 CA 인증서가 등록된 운영체제

4.3 MCP SDK 버전 정책

2026년 7월 10일 기준 MCP TypeScript SDK v2는 베타이며 안정 버전 출시는 2026년 7월 28일로 안내되어 있다. 따라서 초기 운영 버전은 안정적인 v1 계열을 고정하고, v2가 안정화된 후 별도 브랜치에서 마이그레이션을 검증한다.

초기 운영: @modelcontextprotocol/sdk 1.x 버전 고정
 향후 전환: @modelcontextprotocol/server 2.x 안정 버전

버전 범위에 `latest` 또는 무제한 `^`를 사용하지 않고, 검증된 정확한 버전을 사용한다.

5. 전체 구성요소

5.1 redmine-mcp

담당 기능:

- MCP 서버 실행
- Redmine REST API 호출
- 입력값 검증
- Redmine 응답 변환
- 쓰기 작업 확인
- 오류 표준화
- 로그 처리
- 보안 설정 적용

5.2 Claude Code 플러그인

담당 기능:

- Claude Code에 MCP 서버 등록
- Redmine 작업용 스킬 제공
- 사용자 승인 절차 안내
- 플러그인 로컬 테스트 및 배포
- 환경변수와 실행 명령 연결

5.3 Codex 플러그인

담당 기능:

- Codex에 MCP 서버 등록
- Redmine 작업용 스킬 제공
- 사용자 승인 절차 안내
- 로컬 마켓플레이스 등록
- 환경변수와 실행 명령 연결

6. redmine-mcp 기능 요구사항

6.1 연결 확인

RM-FR-001 `redmine_test_connection`

Redmine 서버 연결 및 인증 상태를 확인한다.

입력:

```
{}
```

출력:

```
{
  "connected": true,
  "redmineUrl": "https://redmine.example.com",
  "user": {
    "id": 15,
    "login": "user01",
    "name": "홍길동"
  }
}
```

요구사항:

- `/users/current.json` 을 사용해 인증된 사용자를 확인한다.
- API Key 값은 응답에 포함하지 않는다.
- 연결 실패, 인증 실패, 권한 부족을 구분한다.

Redmine은 `/users/current.json` 을 통해 현재 API 자격 증명의 사용자 정보를 조회할 수 있다.

6.2 프로젝트 조회

RM-FR-002 `redmine_list_projects`

접근 가능한 프로젝트 목록을 조회한다.

입력:

```
{
  "search": "Cloud",
  "limit": 50
}
```

출력 필드:

- 프로젝트 ID
- 프로젝트 식별자
- 프로젝트명
- 설명
- 공개 여부
- 상위 프로젝트
- 활성 상태

요구사항:

- 현재 사용자가 접근 가능한 프로젝트만 반환한다.
- 프로젝트명과 식별자 검색을 지원한다.
- 결과 개수를 제한할 수 있어야 한다.
- 페이지네이션을 처리한다.

Redmine 프로젝트 목록 API는 공개 프로젝트와 현재 사용자에게 접근 권한이 있는 비공개 프로젝트를 반환한다.

6.3 프로젝트 설정 조회

RM-FR-003 `redmine_get_project_context`

이슈 생성과 수정에 필요한 프로젝트 설정을 조회한다.

입력:

```
{
  "projectId": 12
}
```

출력:

- 트래커 목록
- 이슈 상태 목록
- 우선순위 목록
- 카테고리
- 목표 버전
- 프로젝트 멤버
- 사용 가능한 사용자 정의 필드
- 기본 담당자

용도:

- 이름 대신 실제 Redmine ID를 확인
- 사용 가능한 트래커 및 상태 검증
- 이슈 생성 전 선택 항목 제공

6.4 이슈 검색

RM-FR-004 `redmine_search_issues`

조건에 맞는 이슈 목록을 검색한다.

입력 예시:

```
{
  "projectId": 12,
  "assignedTo": "me",
  "status": "open",
  "trackerId": 2,
  "subjectContains": "라이선스",
  "updatedAfter": "2026-07-01T00:00:00+09:00",
  "sort": [
    {
      "field": "updated_on",
      "direction": "desc"
    }
  ],
  "limit": 50
}
```

필수 지원 필터:

- 프로젝트
- 이슈 번호
- 담당자
- 상태

- 트래커
- 우선순위
- 제목 검색
- 생성일
- 수정일
- 상위 이슈
- 사용자 정의 필드
- 열린 이슈·종료 이슈·전체 이슈

요구사항:

- `assignedTo: "me"` 를 지원한다.
- 기본값은 열린 이슈로 설정한다.
- 최대 조회 개수는 서버 설정으로 제한한다.
- Redmine의 최대 페이지 크기인 100개를 초과할 경우 내부적으로 페이지네이션한다.
- 전체 이슈 수, 현재 반환 수, 다음 페이지 존재 여부를 반환한다.

Redmine 이슈 목록 API는 프로젝트, 상태, 담당자, 트래커, 사용자 정의 필드, 생성·수정 날짜 등의 필터와 정렬을 지원한다.

6.5 이슈 상세 조회

RM-FR-005 `redmine_get_issue`

특정 이슈의 상세 내용을 조회한다.

입력:

```
{
  "issueId": 1523,
  "include": [
    "journals",
    "attachments",
    "relations",
    "children",
    "allowed_statuses"
  ]
}
```

출력:

- 이슈 기본 정보
- 설명
- 프로젝트
- 트래커
- 상태
- 우선순위
- 작성자

- 담당자
- 시작일·완료 예정일
- 진척도
- 예상 시간
- 사용자 정의 필드
- 댓글 및 변경 이력
- 첨부 파일 메타데이터
- 관련 이슈
- 하위 이슈
- 현재 사용자가 변경할 수 있는 상태

Redmine 이슈 상세 API는 journals, attachments, relations, children 및 사용자에게 허용된 상태 등을 선택적으로 포함할 수 있다.

6.6 이슈 생성

RM-FR-006 `redmine_create_issue`

신규 이슈를 생성한다.

입력 예시:

```
{
  "projectId": 12,
  "trackerId": 2,
  "subject": "라이선스 만료 알림 기능 추가",
  "description": "라이선스 만료 전에 고객에게 안내 메일을 발송한다.",
  "priorityId": 2,
  "assignedToId": 15,
  "parentIssueId": 1400,
  "fixedVersionId": 8,
  "estimatedHours": 16,
  "customFields": [
    {
      "id": 3,
      "value": "Cloud HMI"
    }
  ],
  "confirm": false
}
```

요구사항:

- `confirm` 기본값은 `false` 다.
- `confirm=false` 일 경우 생성하지 않고 생성 예정 내용을 반환한다.
- `confirm=true` 일 경우에만 실제 API를 호출한다.
- 제목은 필수다.

- 프로젝트와 트래커의 유효성을 검사한다.
- 사용자 정의 필드 타입을 가능한 범위에서 검증한다.
- 성공 시 생성된 이슈 번호와 URL을 반환한다.
- 동일 요청의 중복 실행을 방지할 수 있도록 선택적으로 `clientId` 를 지원한다.

Redmine은 `POST /issues.json` 으로 프로젝트, 트래커, 상태, 우선순위, 제목, 설명, 담당자, 상위 이슈 및 사용자 정의 필드를 포함한 이슈 생성을 지원한다.

6.7 이슈 수정

RM-FR-007 `redmine_update_issue`

기존 이슈의 속성을 수정한다.

지원 항목:

- 제목
- 설명
- 트래커
- 상태
- 우선순위
- 담당자
- 목표 버전
- 카테고리
- 시작일
- 완료 예정일
- 진척도
- 예상 시간
- 상위 이슈
- 사용자 정의 필드

입력 예시:

```
{
  "issueId": 1523,
  "changes": {
    "priorityId": 3,
    "assignedToId": 18,
    "doneRatio": 50
  },
  "comment": "1차 구현을 완료했습니다.",
  "confirm": false
}
```

요구사항:

- 변경 전 값과 변경 후 값을 미리 보여준다.
- 변경 가능한 필드만 허용한다.

- 빈 `changes` 요청을 거부한다.
- 현재 사용자에게 권한이 없으면 실행하지 않는다.
- `confirm=true` 일 경우에만 실제 수정한다.

6.8 댓글 추가

RM-FR-008 `redmine_add_comment`

이슈에 댓글을 추가한다.

입력:

```
{
  "issueId": 1523,
  "comment": "API 구현과 단위 테스트를 완료했습니다.",
  "private": false,
  "confirm": false
}
```

요구사항:

- 공백 댓글은 허용하지 않는다.
- 비공개 댓글은 Redmine 권한이 있을 때만 사용할 수 있다.
- 댓글 내용 미리보기를 제공한다.
- `confirm=true` 일 때만 등록한다.
- 성공 후 등록된 댓글과 이슈 URL을 반환한다.

Redmine은 이슈 수정 요청의 `notes` 필드를 사용하여 댓글을 추가하며 `private_notes` 로 비공개 댓글 여부를 지정할 수 있다.

6.9 상태 변경

RM-FR-009 `redmine_change_status`

이슈 상태를 변경한다.

입력:

```
{
  "issueId": 1523,
  "statusId": 5,
  "comment": "개발 및 테스트가 완료되었습니다.",
  "confirm": false
}
```

요구사항:

- 이슈 상세의 `allowed_statuses`를 확인한다.
- 현재 사용자가 변경할 수 없는 상태는 거부한다.
- 변경 전 상태와 변경 후 상태를 표시한다.
- 필요하면 댓글을 함께 등록한다.
- `confirm=true` 일 때만 상태를 변경한다.

6.10 담당자 변경

RM-FR-010 `redmine_assign_issue`

이슈 담당자를 변경한다.

입력:

```
{
  "issueId": 1523,
  "assignedToId": 18,
  "comment": "백엔드 구현 담당자로 변경합니다.",
  "confirm": false
}
```

요구사항:

- 해당 사용자가 프로젝트 멤버인지 검사한다.
- 담당자 이름만 전달된 경우 후보 목록을 먼저 반환한다.
- 동명이인이 존재하면 자동 선택하지 않는다.
- `confirm=true` 일 때만 변경한다.

7. MVP 제외 기능

다음 기능은 초기 MVP에 포함하지 않는다.

- 이슈 삭제
- 프로젝트 생성·수정·삭제
- Redmine 사용자 생성·삭제
- 관리자 계정 impersonation
- 대량 이슈 일괄 수정
- Wiki 수정
- 저장소 commit 직접 수정
- 첨부 파일 업로드
- 시간 기록 등록
- Webhook 수신
- 원격 HTTP MCP 서버

- OAuth 인증

특히 이슈 삭제와 관리자 기능은 영향도가 크므로 별도 기능 승인 후 추가한다.

8. 쓰기 작업 안전 요구사항

8.1 미리보기 우선

모든 쓰기 도구는 다음과 같이 동작해야 한다.

1차 호출: `confirm=false`

→ 변경 예정 내용 반환

→ 실제 Redmine 데이터 변경 없음

사용자 승인

2차 호출: `confirm=true`

→ 실제 Redmine API 호출

8.2 명시적 승인

다음과 같은 사용자 표현을 승인으로 인정한다.

등록해줘.

진행해.

승인해.

이대로 생성해.

실제로 반영해.

다음 표현만으로는 승인된 것으로 보지 않는다.

어떻게 될까?

초안 작성해줘.

등록할 내용 보여줘.

수정하면 어떻게 돼?

8.3 변경 범위 제한

- 사용자가 요청하지 않은 필드를 자동으로 변경하지 않는다.
- 상태 변경과 담당자 변경을 묶어서 실행하지 않는다.
- 여러 변경을 묶을 경우 전체 변경 내용을 한 번에 보여준다.
- 이슈 설명 전체를 수정할 때 기존 설명을 실수로 제거하지 않도록 한다.
- 댓글 추가와 설명 변경을 구분한다.

8.4 감사 로그

다음 정보만 기록한다.

- 실행 시각
- 도구명
- Redmine URL의 호스트명
- 사용자 ID
- 대상 이슈 번호
- 성공·실패 여부
- HTTP 상태 코드
- 처리 시간
- 오류 유형

다음 정보는 기록하지 않는다.

- API Key
- Authorization 헤더
- 전체 이슈 설명
- 댓글 전체 내용
- 개인정보가 포함된 사용자 정의 필드
- 환경변수 전체 값

9. Redmine API Client 요구사항

9.1 인증

기본 인증 방식:

```
X-Redmine-API-Key: ${REDMINE_API_KEY}
```

금지 사항:

- URL query parameter에 API Key 전달 금지
- 저장소에 API Key 저장 금지
- 플러그인 manifest에 API Key 저장 금지
- 로그에 API Key 출력 금지
- 오류 응답에 API Key 포함 금지

9.2 요청 헤더

```
Accept: application/json
Content-Type: application/json
User-Agent: redmine-mcp/<version>
X-Redmine-API-Key: <secret>
```

Redmine의 POST 및 PUT JSON 요청은 `Content-Type: application/json` 을 명시해야 한다.

9.3 타임아웃

기본값:

연결 타임아웃: 5초
전체 요청 타임아웃: 15초

환경변수로 조정 가능해야 한다.

9.4 재시도

자동 재시도 대상:

- 네트워크 일시 오류
- HTTP 429
- HTTP 502
- HTTP 503
- HTTP 504

정책:

- 최대 2회 재시도
- 지수 백오프 적용
- GET 요청에 우선 적용
- POST와 PUT은 중복 처리 위험이 있으므로 기본적으로 자동 재시도하지 않는다.

9.5 오류 표준화

상황	MCP 오류 코드
Redmine 접속 실패	REDMINE_CONNECTION_ERROR
인증 실패	REDMINE_AUTHENTICATION_ERROR
권한 부족	REDMINE_PERMISSION_DENIED
이슈 없음	REDMINE_ISSUE_NOT_FOUND
프로젝트 없음	REDMINE_PROJECT_NOT_FOUND
입력값 오류	REDMINE_VALIDATION_ERROR
Redmine 422 응답	REDMINE_UNPROCESSABLE_ENTITY
요청 시간 초과	REDMINE_TIMEOUT
인증서 오류	REDMINE_TLS_ERROR
알 수 없는 응답	REDMINE_UNKNOWN_ERROR

오류 메시지는 사용자에게 다음 내용을 제공한다.

- 무엇이 실패했는지
- 변경이 실제로 반영되었는지
- 사용자가 확인해야 할 항목
- 재시도가 안전한지

10. 환경변수 요구사항

10.1 필수 환경변수

```
REDMINE_URL=https://redmine.example.com
REDMINE_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

10.2 선택 환경변수

```
REDMINE_DEFAULT_PROJECT_ID=12
REDMINE_REQUEST_TIMEOUT_MS=15000
REDMINE_CONNECT_TIMEOUT_MS=5000
REDMINE_MAX_RESULT_COUNT=100
REDMINE_LOG_LEVEL=info
REDMINE_CA_CERT_PATH=C:/certs/company-ca.pem
REDMINE_ALLOWED_HOSTS=redmine.example.com
```

10.3 금지 환경변수

다음과 같은 설정은 운영 환경에서 제공하지 않는다.

```
REDMINE_DISABLE_SSL_VERIFY=true
REDMINE_ALLOW_ANY_HOST=true
```

사실 인증서 문제는 SSL 검증 비활성화가 아니라 `REDMINE_CA_CERT_PATH` 로 해결한다.

11. MCP 서버 요구사항

11.1 전송 방식

초기 버전은 STDIO만 지원한다.

Codex 또는 Claude Code

→ redmine-mcp 프로세스 실행

→ stdin/stdout으로 MCP 메시지 송수신

11.2 출력 규칙

- MCP 프로토콜 메시지만 stdout에 출력한다.
- 일반 로그는 stderr에 출력한다.
- 디버그 로그도 stdout에 출력하지 않는다.
- 모든 도구 입력은 스키마로 검증한다.
- 예상하지 못한 필드는 기본적으로 거부한다.

공식 MCP SDK 가이드에서도 STDIO 서버의 stdout은 프로토콜에 사용하고 로그는 stderr에 기록하도록 안내한다.

11.3 도구 명명 규칙

```
redmine_test_connection
redmine_list_projects
redmine_get_project_context
redmine_search_issues
redmine_get_issue
redmine_create_issue
redmine_update_issue
redmine_add_comment
redmine_change_status
redmine_assign_issue
```

규칙:

- 소문자 snake_case 사용
- 이름에 회사 고유 프로젝트명을 넣지 않는다.
- 조회와 변경 작업을 구분할 수 있어야 한다.
- 하나의 도구가 너무 많은 역할을 수행하지 않는다.

11.4 MCP 서버 지침

서버 초기화 시 다음과 같은 공통 지침을 제공한다.

Redmine 조회 도구는 승인 없이 사용할 수 있다.
이슈 생성, 수정, 댓글, 상태 및 담당자 변경은 사용자가 내용을
확인하고 명시적으로 승인한 경우에만 실행한다.
쓰기 작업은 먼저 confirm=false로 미리보기를 생성한다.
API Key나 인증 정보를 출력하지 않는다.

Codex는 MCP 초기화 시 서버가 반환하는 `instructions` 를 도구 사용 지침으로 활용하므로, 핵심 제약사항은 앞 부분에 배치한다.

12. Claude Code 플러그인 요구사항

12.1 디렉터리 구조

```
claude-redmine-plugin/  
├── .claude-plugin/  
│   └── plugin.json  
├── .mcp.json  
├── skills/  
│   ├── redmine-issue-search/  
│   │   └── SKILL.md  
│   ├── redmine-issue-create/  
│   │   └── SKILL.md  
│   ├── redmine-issue-update/  
│   │   └── SKILL.md  
│   └── redmine-work-summary/  
│       └── SKILL.md  
├── README.md  
├── CHANGELOG.md  
└── LICENSE
```

Claude Code에서는 `plugin.json` 만 `.claude-plugin/` 에 두고 `.mcp.json` 과 `skills/` 는 플러그인 루트에 배치해야 한다.

12.2 Manifest 요구사항

```
{  
  "name": "redmine",  
  "description": "Search, create and update Redmine issues from Claude Code.",  
  "version": "0.1.0",  
  "author": {  
    "name": "M2I"  
  },  
  "license": "MIT"  
}
```

요구사항:

- `name` 은 `redmine` 또는 회사에서 승인한 고유 이름으로 한다.
- 버전은 Semantic Versioning을 사용한다.
- `description` 은 영어를 기본으로 한다.
- 사내 전용일 경우 repository 주소는 사내 GitLab을 사용한다.

Claude Code 플러그인 manifest에서 `name` 은 필수이며, 플러그인 이름은 스킬 namespace로 사용된다.

12.3 MCP 설정

```
{
  "mcpServers": {
    "redmine": {
      "command": "npx",
      "args": [
        "-y",
        "@m2i/redmine-mcp@0.1.0"
      ],
      "env": {
        "REDMINE_URL": "${REDMINE_URL}",
        "REDMINE_API_KEY": "${REDMINE_API_KEY}"
      }
    }
  }
}
```

요구사항:

- MCP 패키지 버전을 고정한다.
- 사용자 API Key를 plugin.json에 넣지 않는다.
- 플러그인 활성화 시 MCP 서버가 자동 실행되어야 한다.
- 플러그인 비활성화 시 서버도 실행되지 않아야 한다.
- 변경 후 `/reload-plugins` 로 재로딩 가능해야 한다.

Claude Code 플러그인의 `.mcp.json` 은 표준 `mcpServers` 형식을 사용하며 플러그인이 활성화되면 포함된 MCP 서버가 자동 시작된다.

12.4 Claude Code 스킴

`redmine-issue-search`

역할:

- 자연어 검색 조건을 Redmine 검색 필터로 변환
- 검색 결과를 번호, 제목, 상태, 담당자, 수정일 기준으로 정리

`redmine-issue-create`

역할:

- 현재 대화와 Git 변경사항을 기반으로 이슈 초안 생성
- 제목, 배경, 현재 문제, 기대 결과, 구현 내용, 테스트 조건 작성
- 프로젝트와 트래커가 불명확하면 조회 도구를 먼저 사용
- 실제 생성 전 사용자에게 전체 내용 표시

redmine-issue-update

역할:

- 작업 결과를 기존 이슈 댓글 형식으로 정리
- 상태 변경과 댓글 등록을 구분
- 실제 반영 전 변경 내용을 표시

redmine-work-summary

역할:

- Git diff와 테스트 결과를 요약
- Redmine 댓글에 적합한 형식으로 변환
- 자동 등록하지 않고 초안을 우선 제공

12.5 로컬 테스트

```
claude --plugin-dir ./claude-redmine-plugin
```

검증 항목:

```
/plugin
/mcp
/redmine:redmine-issue-search
/redmine:redmine-issue-create
```

Claude Code는 `--plugin-dir` 로 로컬 플러그인을 테스트할 수 있다.

13. Codex 플러그인 요구사항

13.1 디렉터리 구조

```
codex-redmine-plugin/
├── .codex-plugin/
│   └── plugin.json
├── .mcp.json
├── skills/
│   ├── redmine-issue-search/
│   │   └── SKILL.md
│   ├── redmine-issue-create/
│   │   └── SKILL.md
│   ├── redmine-issue-update/
│   │   └── SKILL.md
```

```

|   └─ redmine-work-summary/
|       └─ SKILL.md
├─ assets/
|   └─ icon.svg
|       └─ logo.png
├─ README.md
├─ CHANGELOG.md
└─ LICENSE

```

Codex 플러그인은 `.codex-plugin/plugin.json` 을 필수 manifest로 사용하고, `skills/`, `.mcp.json`, `assets/` 등을 플러그인 루트에 배치한다.

13.2 Manifest 요구사항

```

{
  "name": "redmine",
  "version": "0.1.0",
  "description": "Search, create and update Redmine issues from Codex.",
  "author": {
    "name": "M2I"
  },
  "license": "MIT",
  "skills": "./skills/",
  "mcpServers": "./.mcp.json",
  "interface": {
    "displayName": "Redmine",
    "shortDescription": "Manage Redmine issues",
    "longDescription": "Search, create, comment on and update Redmine issues from Codex.",
    "developerName": "M2I",
    "category": "Productivity",
    "capabilities": [
      "Read",
      "Write"
    ]
  }
}

```

13.3 MCP 설정

```

{
  "redmine": {
    "command": "npx",
    "args": [
      "-y",
      "@m2i/redmine-mcp@0.1.0"
    ]
  }
}

```

```

],
"env_vars": [
  "REDMINE_URL",
  "REDMINE_API_KEY",
  "REDMINE_CA_CERT_PATH"
]
}
}

```

요구사항:

- `.mcp.json` 은 직접 서버 맵 방식을 우선 사용한다.
- 서버명은 `redmine` 으로 통일한다.
- 환경변수 값 자체를 manifest에 저장하지 않는다.
- 조회 도구는 승인 정책을 완화할 수 있다.
- 쓰기 도구는 기본적으로 사용자 승인 정책을 사용한다.

Codex 플러그인의 `mcpServers` 는 직접 서버 맵 또는 `mcp_servers` 로 감싼 형식을 사용할 수 있으며, 플러그인 별·도구별 승인 정책을 별도로 설정할 수 있다.

13.4 승인 정책 권장값

```

[plugins."redmine".mcp_servers.redmine]
enabled = true
default_tools_approval_mode = "prompt"

[plugins."redmine".mcp_servers.redmine.tools.redmine_test_connection]
approval_mode = "approve"

[plugins."redmine".mcp_servers.redmine.tools.redmine_list_projects]
approval_mode = "approve"

[plugins."redmine".mcp_servers.redmine.tools.redmine_search_issues]
approval_mode = "approve"

[plugins."redmine".mcp_servers.redmine.tools.redmine_get_issue]
approval_mode = "approve"

[plugins."redmine".mcp_servers.redmine.tools.redmine_create_issue]
approval_mode = "prompt"

[plugins."redmine".mcp_servers.redmine.tools.redmine_update_issue]
approval_mode = "prompt"

[plugins."redmine".mcp_servers.redmine.tools.redmine_add_comment]
approval_mode = "prompt"

```

```
[plugins."redmine".mcp_servers.redmine.tools.redmine_change_status]
approval_mode = "prompt"

[plugins."redmine".mcp_servers.redmine.tools.redmine_assign_issue]
approval_mode = "prompt"
```

13.5 로컬 마켓플레이스

저장소 구조:

```
redmine-codex-marketplace/
├── .agents/
│   └── plugins/
│       └── marketplace.json
└── plugins/
    └── redmine/
```

사용자 등록 예시:

```
codex plugin marketplace add ./redmine-codex-marketplace
```

또는 사내 GitLab:

```
codex plugin marketplace add
ssh://git@gitlab.example.com/ai/redmine-codex-marketplace.git
```

Codex는 로컬 디렉터리, GitHub shorthand, HTTPS 또는 SSH Git 저장소를 marketplace 소스로 지원한다.

14. 패키지 및 저장소 구조

14.1 권장 모노레포

```
redmine-agent-integration/
├── packages/
│   ├── redmine-client/
│   │   ├── src/
│   │   └── package.json
│   └── redmine-mcp/
│       ├── src/
│       │   ├── server.ts
│       │   ├── config.ts
│       │   └── errors.ts
```

```

|
|   |
|   |   └─ logging.ts
|   |   └─ tools/
|   |       └─ connection.ts
|   |       └─ projects.ts
|   |       └─ issues.ts
|   └─ tests/
|   └─ package.json
└─ plugins/
    └─ claude-code/
    └─ codex/
└─ marketplaces/
    └─ claude/
    └─ codex/
└─ docs/
    └─ installation.md
    └─ security.md
    └─ troubleshooting.md
    └─ development.md
└─ package.json
└─ pnpm-workspace.yaml
└─ README.md

```

14.2 패키지 배포

권장 패키지명:

```
@m2i/redmine-client
@m2i/redmine-mcp
```

사내 전용일 경우:

- 사내 GitLab Package Registry 또는 Nexus 사용
- 패키지 접근 권한을 사내 계정으로 제한
- npm package integrity 검증
- 배포 버전은 Git tag와 일치
- 운영 버전은 pre-release 의존성을 사용하지 않음

15. 보안 요구사항

15.1 API Key 보호

- 개인별 API Key 사용
- 공용 관리자 API Key 사용 금지
- API Key를 Git에 commit하지 않음
- `.env` 파일은 `.gitignore`에 포함

- API Key를 명령행 인자로 전달하지 않음
- 프로세스 목록에 API Key가 노출되지 않도록 환경변수 사용
- 오류 및 디버그 로그에서 secret masking 적용

15.2 Redmine 호스트 제한

`REDMINE_ALLOWED_HOSTS` 가 설정된 경우 해당 호스트만 허용한다.

예:

```
REDMINE_ALLOWED_HOSTS=redmine.example.com,redmine-dev.example.com
```

다음 URL은 거부한다.

```
http://127.0.0.1
http://169.254.169.254
file://
ftp://
임의의 사용자 입력 URL
```

목적:

- SSRF 방지
- 클라우드 메타데이터 접근 방지
- 다른 내부 서비스 접근 방지

15.3 권한 최소화

- 관리자 계정 사용을 요구하지 않는다.
- 사용자 본인의 API Key를 사용한다.
- Redmine이 반환한 권한 오류를 우회하지 않는다.
- 사용자 impersonation 기능을 구현하지 않는다.
- 플러그인이 OS 관리자 권한을 요구하지 않는다.

15.4 입력값 제한

- 이슈 번호는 양의 정수만 허용
- `limit` 상한 설정
- 설명 및 댓글 최대 길이 설정
- URL은 환경 설정에서만 입력
- 파일 경로는 허용된 CA 인증서 경로에만 사용
- 프로토타입 오염 방지를 위해 JSON 스키마에 예상 필드만 허용

16. 비기능 요구사항

16.1 성능

- MCP 서버 시작 시간: 2초 이내
- 일반 이슈 조회 응답: 3초 이내
- Redmine API 응답이 느린 경우 진행 상태 또는 명확한 대기 메시지 제공
- 100건 조회 시 불필요한 개별 상세 API 호출 금지
- 프로젝트 설정은 짧은 TTL로 메모리 캐시 가능

16.2 안정성

- Redmine 연결 실패가 MCP 프로세스 종료로 이어지지 않아야 한다.
- 하나의 도구 호출 실패가 다음 호출에 영향을 주지 않아야 한다.
- 처리하지 못한 예외는 표준 MCP 오류로 변환한다.
- STDIO 연결 종료 시 프로세스를 정상 종료한다.
- SIGINT와 SIGTERM을 처리한다.

16.3 호환성

지원 대상:

- Windows PowerShell
- Windows WSL2
- macOS
- Ubuntu Linux
- Codex CLI
- Codex IDE Extension
- Claude Code CLI

경로 구분자와 shell quoting 차이를 고려한다.

16.4 유지보수성

- Redmine API Client와 MCP Tool 계층 분리
 - 도구별 파일 분리
 - 입력·출력 타입 정의
 - API 응답 DTO 정의
 - eslint 및 formatter 적용
 - 최소 80% 이상의 핵심 로직 테스트 커버리지
 - Conventional Commits 또는 사내 커밋 규칙 적용
-

17. 테스트 요구사항

17.1 단위 테스트

테스트 대상:

- 환경변수 파싱
- URL 검증
- API Key masking
- 입력 스키마 검증
- Redmine 오류 변환
- 페이지네이션
- 날짜 필터 변환
- 확인 절차
- 이슈 변경 diff 생성
- 응답 데이터 정규화

17.2 통합 테스트

별도 테스트용 Redmine을 사용한다.

필수 시나리오:

1. 정상 인증
2. 잘못된 API Key
3. 권한 없는 프로젝트
4. 이슈 목록 조회
5. 100건 초과 페이지네이션
6. 이슈 상세 및 댓글 조회
7. 이슈 생성 미리보기
8. 이슈 실제 생성
9. 댓글 추가
10. 허용된 상태 변경
11. 허용되지 않은 상태 변경
12. 담당자 변경
13. Redmine 422 오류
14. 연결 시간 초과
15. 사설 CA 인증서 적용

17.3 MCP 호환성 테스트

- MCP Inspector에서 전체 도구 목록 확인
- 각 도구 input schema 확인
- 잘못된 입력 거부 확인
- stdout에 로그가 섞이지 않는지 확인
- Codex에서 도구 목록 확인
- Claude Code에서 도구 목록 확인

17.4 플러그인 테스트

Claude Code

```
claude --plugin-dir ./plugins/claude-code
```

확인 사항:

- 플러그인 인식
- MCP 서버 자동 실행
- 스킬 namespace
- 환경변수 누락 오류
- `/reload-plugins` 적용
- 쓰기 작업 승인

Codex

확인 사항:

- 로컬 marketplace 인식
- 플러그인 설치·활성화
- `.mcp.json` 로딩
- `/mcp` 연결 상태
- 조회 도구 자동 실행
- 쓰기 도구 승인 표시

18. 설치 문서 요구사항

README에 다음 내용을 포함한다.

1. 기능 소개
2. 지원 환경
3. Redmine REST API 활성화 방법
4. API Key 확인 방법
5. 환경변수 설정
6. Claude Code 설치 방법
7. Codex 설치 방법
8. 연결 확인 방법
9. 사용 예제
10. 쓰기 작업 승인 방식
11. 사내 인증서 설정
12. 오류 해결 방법
13. 보안 주의사항
14. 제거 방법
15. 라이선스

API Key 실제 값이 포함된 화면이나 예제는 사용하지 않는다.

19. 완료 조건

19.1 redmine-mcp 완료 조건

- ☐ Redmine 연결 확인 가능
- ☐ 접근 가능한 프로젝트 조회 가능
- ☐ 이슈 검색 가능
- ☐ 이슈 상세 및 댓글 조회 가능
- ☐ 이슈 생성 가능
- ☐ 이슈 수정 가능
- ☐ 댓글 추가 가능
- ☐ 상태 변경 가능
- ☐ 담당자 변경 가능
- ☐ 모든 쓰기 작업에 미리보기와 승인 적용
- ☐ API Key가 로그에 노출되지 않음
- ☐ Windows, macOS, Linux에서 실행 가능
- ☐ MCP Inspector 테스트 통과

19.2 Claude Code 플러그인 완료 조건

- ☐ `--plugin-dir` 로 로컬 실행 가능
- ☐ 플러그인 활성화 시 MCP 서버 연결
- ☐ Redmine 관련 스킬 4종 제공
- ☐ 조회와 쓰기 작업이 구분됨
- ☐ `/reload-plugins` 후 변경사항 적용
- ☐ 사내 marketplace 또는 Git 저장소에서 설치 가능

19.3 Codex 플러그인 완료 조건

- ☐ `.codex-plugin/plugin.json` 검증 통과
- ☐ 로컬 marketplace에서 검색 및 설치 가능
- ☐ 플러그인 활성화 시 MCP 서버 연결
- ☐ 조회 도구와 쓰기 도구 승인 정책 분리
- ☐ Codex CLI 및 IDE Extension에서 동일하게 사용 가능
- ☐ 사내 GitLab marketplace에서 배포 가능

20. 개발 단계

1단계: 조회 전용 MVP

구현 기능:

- 연결 확인
- 프로젝트 조회
- 이슈 검색
- 이슈 상세 조회

- Claude Code 로컬 연결
- Codex 로컬 연결

이 단계에서는 Redmine 데이터를 변경하지 않는다.

2단계: 제한된 쓰기 기능

추가 기능:

- 이슈 생성
- 댓글 추가
- 상태 변경
- 미리보기와 승인
- 감사 로그

3단계: 개발 워크플로 연동

추가 기능:

- Git diff 기반 이슈 초안
- 테스트 결과 댓글 작성
- 커밋 메시지에서 이슈 번호 추출
- 브랜치명과 이슈 번호 연결
- 작업 완료 요약

4단계: 사내 정식 배포

추가 작업:

- 사내 npm 레지스트리 배포
- Claude Code marketplace 배포
- Codex marketplace 배포
- 보안 검토
- 운영 문서 작성
- 버전 업데이트 정책 수립